Reporte Técnico de Actividades Práctico-Experimentales Nro. 002

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Steeven Daniel Pardo Lima Royel Ivan Jima Pardo Mark Anthony Gonzalez Jaramillo Juan Alberto Calopino Martinez Servio Julian Vega Jimenez
Asignatura	Desarrollo Basado en Plataformas
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	002
Título de la Práctica	Implementar un servicio REST con Node.js
Nombre del Docente	Edison Leonardo Coronel Romero
Fecha	Viernes 3 de octubre
Horario	11h30 – 13H30
Lugar	Laboratorio de redes y Sistemas Operativos
Tiempo planificado en el Sílabo	3 horas
Modalidad de trabajo	Cada estudiante simula en su computadora; se trabaja y se entrega el trabajo grupal; la prueba física se hace con la placa del grupo.

2. Objetivo(s) de la Práctica

Objetivo. Programar en el ESP32 un juego de luces con cuatro LEDs cuya velocidad cambia con un pulsador, usando temporización no bloqueante con millis() y la lectura del pulsador con pull-up interna, antirrebote y detección de flanco.

3. Materiales, Reactivos, Equipos y Herramientas

Hardware (por grupo, solo para la Parte C):

Cantidad	Componente
1	ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga)
1	Protoboard

4	LEDs de 5 mm rojos, amarillos o verdes (no azules ni blancos: casi no encienden a 3,3 V)
4	Resistencias de 330 Ω
1	Pulsador de 4 patas
6	Cables jumper macho-macho

Software (cada estudiante):

- Wokwi (wokwi.com), en el navegador, sin instalar nada.
- Arduino IDE 2.x con el paquete esp32 by Espressif Systems versión 3.x, para la Parte C.
- Cuenta de GitHub y acceso al EVA, para la Parte D.

4. Procedimiento / Metodología

1. Repartimos las tareas entre los cinco miembros del grupo con el fin de que unos participantes realicen la simulación y en base a ello los demás puedan replicarla en la práctica real.

2. Entre los miembros del grupo se validó que tanto la simulación y lo que se armó en físico sea el correcto.

5. Resultados

Simulación en WOKWI

Link de repositorios Woki:
Simulación parte A: Ingrese aquí
Simulación parte B: Ingrese aquí

Parte A. Circuito y secuencia de luces (Wokwi)

- Simulación Parte 1.

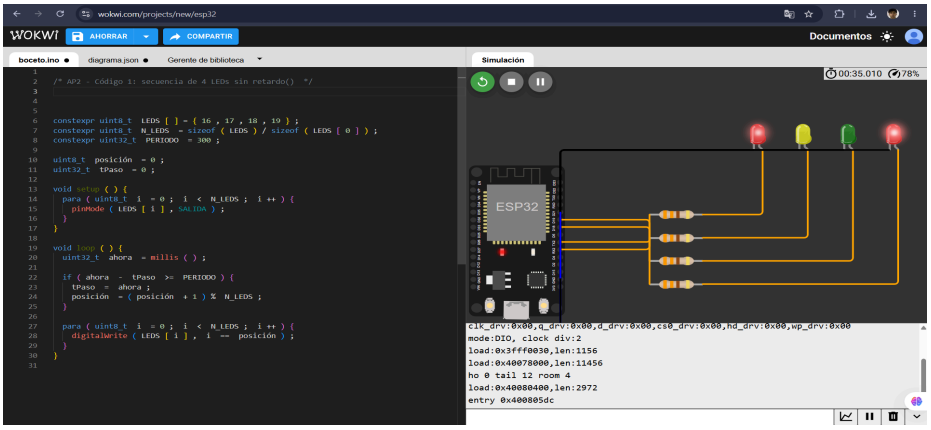


Imagen 1: Tabla A con 300ms

Se ejecutó el programa, la luz avanzará del LED 1 al 4 y regresará al primero, cambiando cada 300 milisegundos.

- Simulación parte 2.

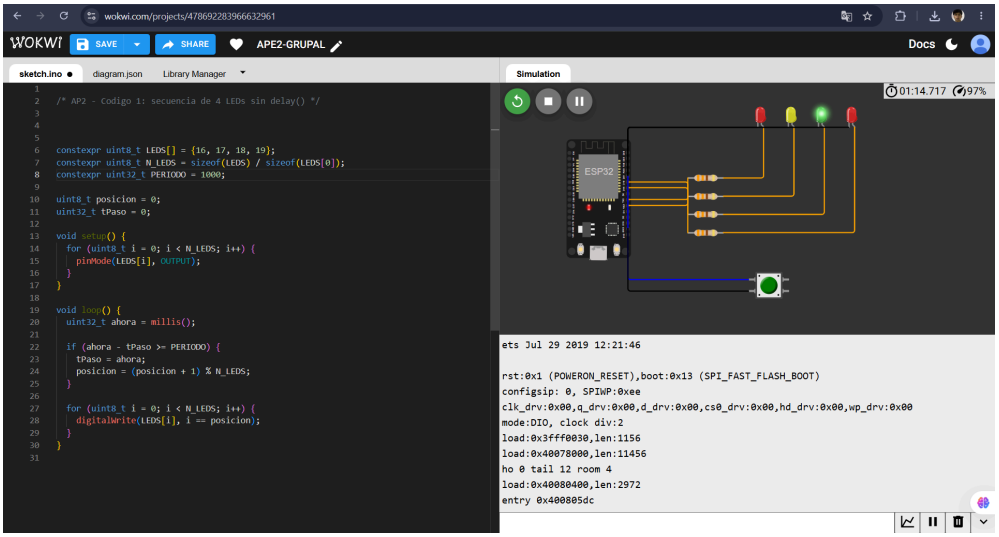


Imagen 2: Tabla A con 1000ms

Se ejecutó el programa, la luz avanzará del LED 1 al 4 y regresará al primero, cambiando cada 1000 milisegundos.

- Simulación parte 3

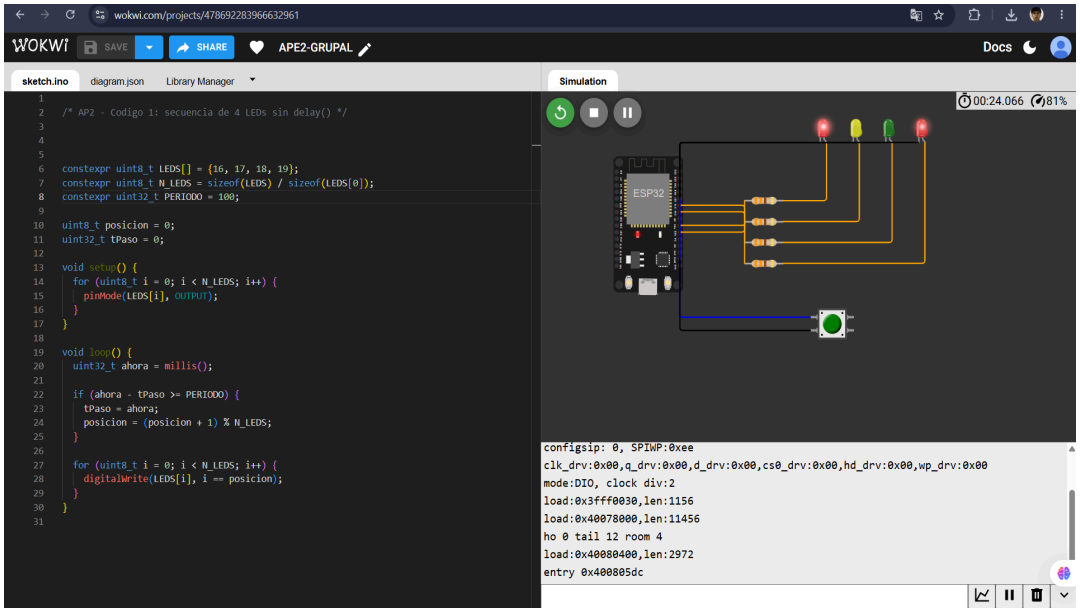


Imagen 3: Tabla A con 100ms

Se ejecutó el programa, la luz avanzará del LED 1 al 4 y regresará al primero, cambiando cada 100 milisegundos.

Tabla A. Efecto del periodo

Periodo(ms)	¿Qué observan?
100	Secuencia de los led rápida, cambia cada 0.1 segundos.
300	Secuencia de los led media, cambia cada 0.3 segundos.
1000	Secuencia de los led lenta, cambia cada 1 segundo.

Parte B. El pulsador cambia la velocidad (Wokwi)

- Simulación Parte 1.

Se ejecutó el programa añadiendo el pulsador para cambiar la velocidad con un T_REBOTE = 30ms.

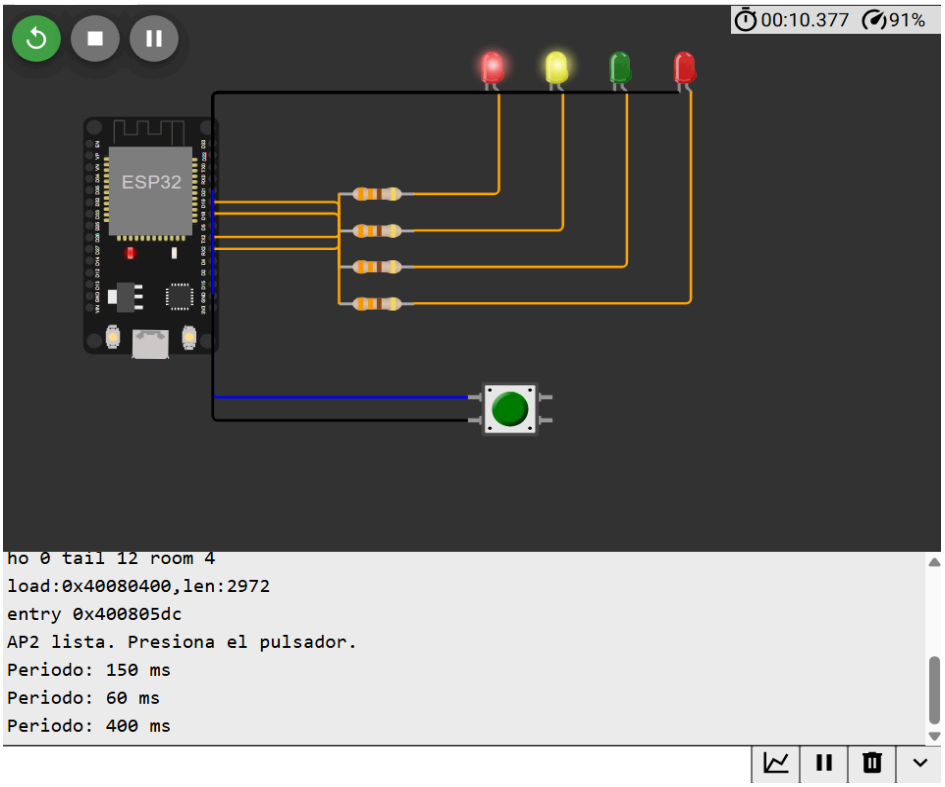


Imagen 4. Tabla B con 30 T_REBOTE

- Simulación Parte 2.

Se ejecutó el programa cambiando el T_REBOTE del pulsador de 30ms a 0 ms.

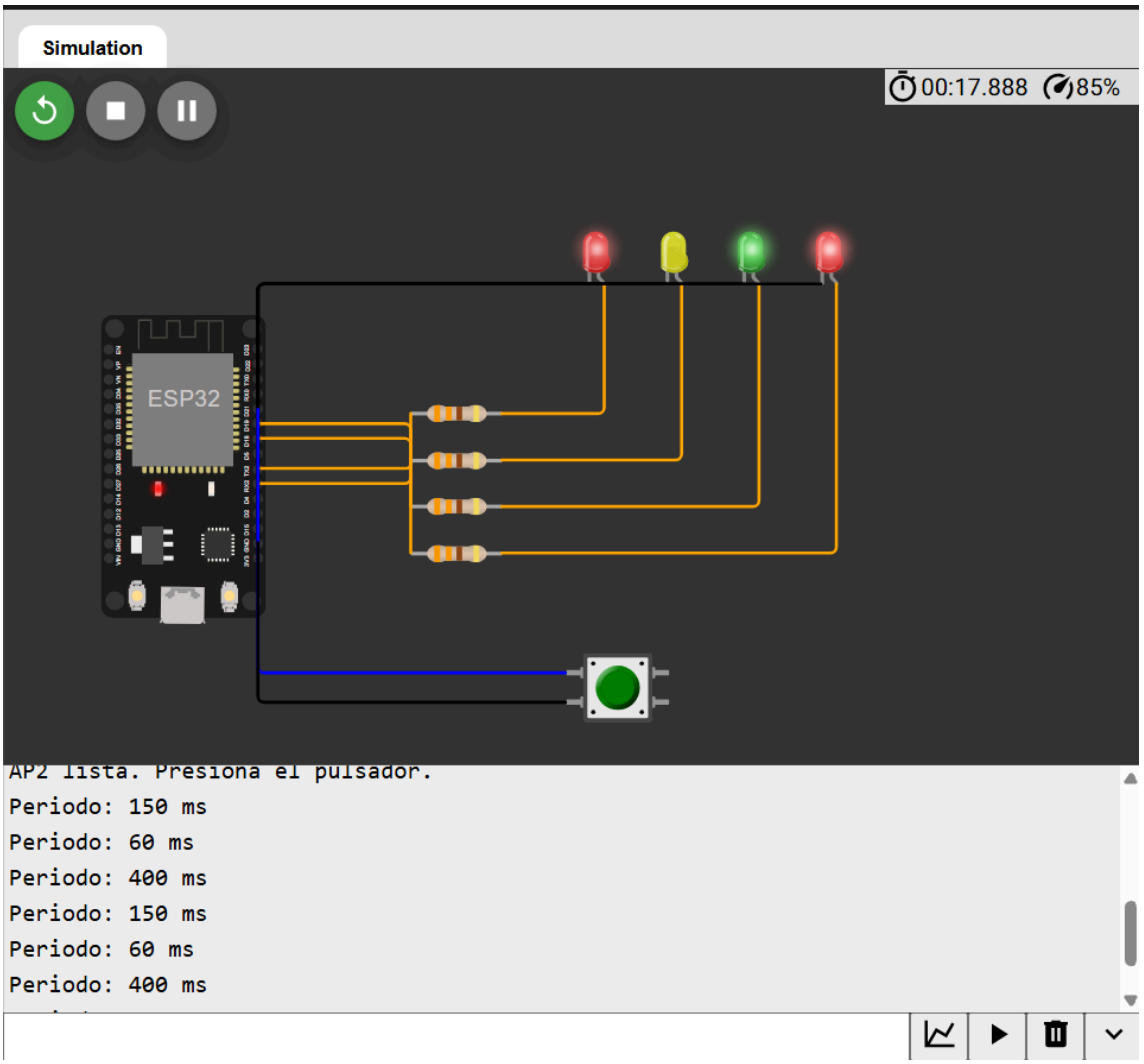


Imagen 5. Tabla B con 0 T_REBOTE

Tabla B. Efecto del antirrebote (10 clicks)

T_REBOTE (ms)	Líneas impresas	¿Coincide con los 10 clics?
0	Imprimió 72 líneas de cambio de velocidad con el pulsador.	No coincide. Al desactivarse el filtrado, cada rebote mecánico del contacto que dura entre 1 y 5 ms se puede

		interpretar como una pulsación nueva, de modo que el programa cuenta pulsaciones que nunca ocurrieron y el Monitor Serie imprime más líneas que clics realizados.
30	Imprimió correctamente las 10 líneas del cambio de velocidad.	Sí. El antirrebote exige que la señal se mantenga estable 30 ms antes de aceptar el cambio; los rebotes reales duran solo 1 a 5 ms y quedan filtrados, por lo que cada clic produce una única línea.

Parte C. Prueba en la placa ESP32 real

- Simulación Parte 1 real.

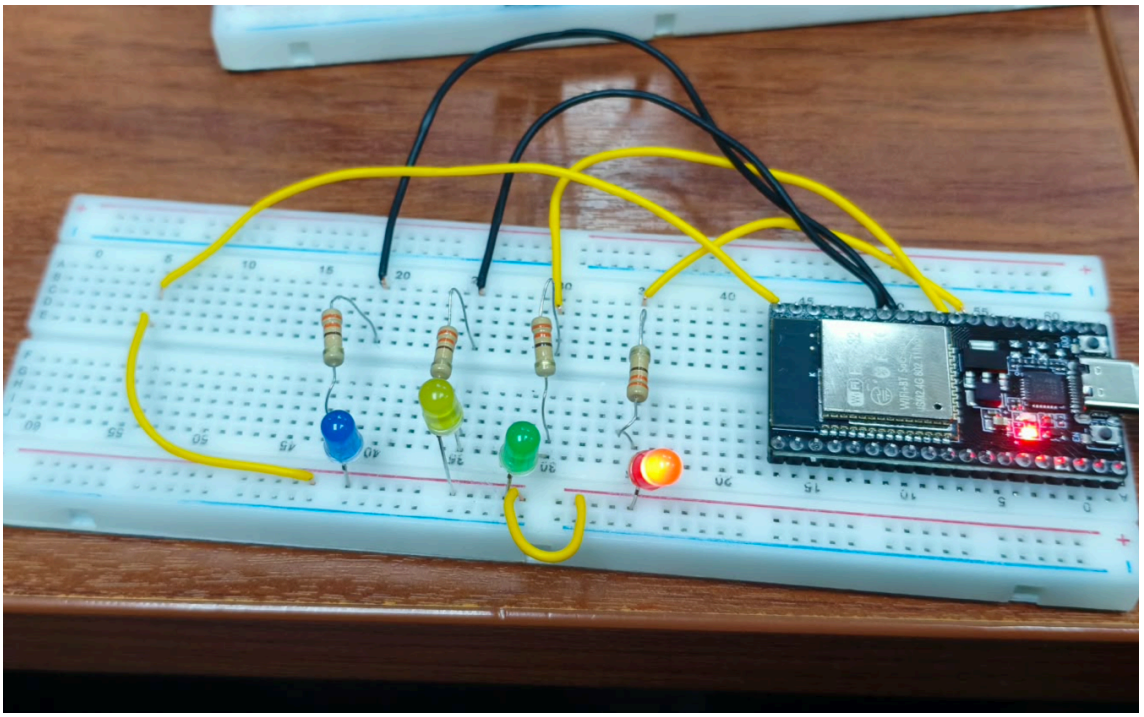


Imagen 6. Repetición de los pasos b2

- Simulación Parte 2 real.

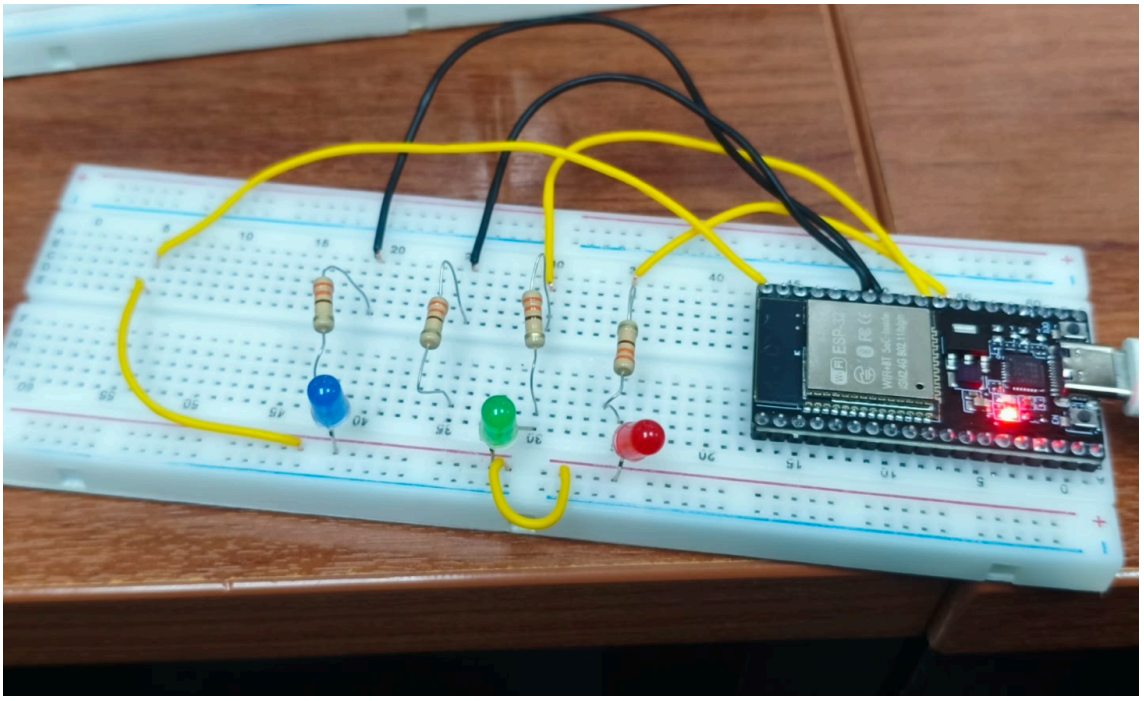


Imagen 7. Repetición de los pasos b3

Parte D. Entrega

Link del GitHub:

https://github.com/JuanCalopinoo/APE2-SistemasDigitales-Jima-Vega-Calopino-Gonzalez-Pardo.git

Link del funcionamiento en Drive:

6. Preguntas de Control

1. ¿Por qué el Código 1 usa millis() en lugar de delay()? ¿Qué pasaría en el Código 2 si cada paso usara delay(400)?

millis() permite una temporización no bloqueante: loop() nunca se detiene, solo compara el tiempo transcurrido (ahora − tPaso >= PERIODO) y, mientras no se cumpla, sigue ejecutándose y puede leer entradas miles de veces por segundo. En cambio, delay(400) congela por completo el programa durante esos 400 ms. Si cada paso de la secuencia usara delay(400), el pulsador solo se revisaría una vez cada 400 ms: las pulsaciones cortas se perderían, la respuesta se sentiría lenta y desincronizada, y el antirrebote y la detección de flanco dejarían de ser fiables porque el código de lectura del botón casi no llegaría a ejecutarse.

2. El pulsador va entre el GPIO21 y GND, sin resistencia externa. ¿Qué valor lee digitalRead() con el pulsador suelto y con el pulsador presionado? ¿Por qué?

Con el pulsador suelto, digitalRead() devuelve HIGH (1); con el pulsador presionado devuelve LOW (0). La razón es que pinMode(21, INPUT_PULLUP) activa la resistencia pull-up interna del ESP32, que conecta suavemente el pin a 3,3 V, fijando un nivel definido cuando no hay nada presionado. Al presionar el botón, el pin se une directamente a GND y el nivel cae a 0 V. Si no existiera esa pull-up (y tampoco una externa), el pin quedaría flotando y daría lecturas aleatorias; por eso el circuito no necesita ninguna resistencia adicional.

3. ¿Qué pasaría si conectaran el pulsador al GPIO34 sin cambiar el código?

Los GPIO 34 a 39 del ESP32 son pines de solo entrada y, a diferencia del resto, no tienen resistencia pull-up ni pull-down interna activable. Al llamar pinMode(34, INPUT_PULLUP) esa pull-up simplemente no se activa, de modo que con el pulsador suelto el pin queda flotando y digitalRead() devuelve valores aleatorios (a veces HIGH, a veces LOW) por ruido eléctrico. El programa interpretaría esas variaciones como pulsaciones falsas y el juego cambiaría de velocidad solo, sin que nadie toque el botón. La solución sería añadir una resistencia externa de 10 kΩ a 3,3 V o, mejor, usar un pin normal como el GPIO21 propuesto en la guía.

4. En el paso B3 la velocidad cambió una sola vez aunque mantuvieron presionado el pulsador. ¿Qué parte del Código 2 lo garantiza?

Lo garantiza la función botonPresionado(), concretamente su detección de flanco: la función solo devuelve true en el instante en que el estado filtrado (estadoEstable) pasa de HIGH a LOW, es decir, en el flanco de bajada. Mientras el botón siga presionado, la lectura es igual a estadoEstable, la condición lectura != estadoEstable no se vuelve a cumplir y la función retorna false, así que no hay más cambios. El antirrebote de T_REBOTE = 30 ms complementa esta lógica asegurando que ese único flanco aceptado corresponda a un cambio real de la señal y no a un rebote mecánico.

5. Expliquen la Tabla B: ¿por qué con T_REBOTE = 0 puede haber más líneas impresas que clics?

Con T_REBOTE = 0 el programa acepta cualquier transición de la señal sin filtrar. Los contactos metálicos de un pulsador real rebotan durante unos milisegundos al presionarlo o soltarlo: la señal rebota varias veces entre HIGH y LOW antes de estabilizarse, y cada uno de esos flancos de bajada se interpreta como una pulsación nueva, imprimiendo líneas adicionales. Por eso con 10 clics pueden aparecer más de 10 líneas, especialmente en la placa física. En Wokwi con clics pausados puede coincidir, porque el botón simulado es ideal; con 30 ms de antirrebote la señal debe permanecer estable ese tiempo para aceptarse, los rebotes (de 1 a 5 ms) se descartan y el conteo coincide exactamente con los clics.

6. Calculen la corriente que circula por un LED rojo (Vf ≈ 2,0 V) con una resistencia de 330 Ω a 3,3 V. ¿Por qué no conviene usar un LED azul?

Aplicando la Ley de Ohm sobre la tensión que cae en la resistencia: I = (Vs − Vf) / R = (3,3 V − 2,0 V) / 330 Ω = 1,3 V / 330 Ω ≈ 3,9 mA. Esa corriente es suficiente para que un LED rojo, amarillo o verde se vea claramente. Un LED azul tiene una tensión directa mucho mayor (Vf ≈ 3,0 a 3,2 V), de modo que a 3,3 V solo queda un margen de 0,1 a

0,3 V: la corriente sería de apenas 0,3 a 0,9 mA y el LED casi no encendería; además, cualquier variación de la alimentación puede dejarlo directamente sin conducir. Por eso la guía pide explícitamente no usar LEDs azules ni blancos (el blanco tiene un Vf similar al azul).

7. Mencionen una diferencia entre probar el circuito en Wokwi y probarlo en la placa real.

En Wokwi el circuito es ideal: no hay rebotes mecánicos reales, ni ruido eléctrico, ni riesgo de invertir la polaridad de un LED o de dañar la placa, y cualquier error de cableado se corrige con un clic. En la ESP32 real aparecen esos efectos físicos: los rebotes del pulsador son verdaderos (se nota en la Tabla B), un LED conectado al revés sencillamente no enciende, y cargar el programa depende del cable USB de datos, del driver (CP210x o CH340) y del puerto COM/ttyUSB0 correcto. Por eso el comportamiento medido en la placa puede diferir levemente del simulador, aunque el código sea el mismo.

8. El ESP32 sigue funcionando a 240 MHz aunque el juego solo cambie de LED cada 400 ms. Propongan una idea para reducir el consumo de energía si el juego funcionara con batería.

La idea más efectiva es aprovechar el modo de bajo consumo deep sleep: cuando el juego queda inactivo unos segundos, el ESP32 se apaga casi por completo. Al despertar, enciende la secuencia y vuelve a dormir si no hay actividad. Como medidas complementarias se puede reducir la frecuencia de la CPU con setCpuFrequencyMhz(80) en lugar de 240 MHz, o bajar el brillo de los LEDs usando PWM con un duty bajo, ya que los LEDs son el elemento que más corriente consume del circuito.

7. Conclusiones

- La temporización no bloqueante con `millis()` permite que el ESP32 haga varias cosas a la vez. Al no detener `loop()`, el sistema anima los LEDs y a la vez atiende el pulsador sin perder pulsaciones, algo imposible con `delay()` sin sacrificar la respuesta.
- El antirrebote por software es indispensable al leer pulsadores mecánicos. La Tabla B muestra que sin filtro (`T_REBOTE = 0`) una sola pulsación puede interpretarse como varias, y que un tiempo de estabilización de 30 ms garantiza una lectura por clic.
- La detección de flanco, junto con la pull-up interna, simplifica el hardware y la lógica. Con `INPUT_PULLUP` el pulsador se conecta directo entre el GPIO y GND sin resistencia externa. La detección de flanco asegura que una pulsación sostenida produzca un único cambio de velocidad.

8. Recomendaciones

- Verificar la serigrafía de la placa antes de cablear: en los clones el orden de los pines (RX2, TX2, D18, D19, D21) puede variar. Comparar siempre con la tabla de conexiones.
- Revisar la polaridad de los LEDs (pata larga al ánodo, hacia la resistencia y el GPIO) y no omitir las resistencias de 330 Ω.
- Usar un cable USB de datos (no solo de carga). Si la carga se queda en `Connecting...`, mantener presionado el botón BOOT; si el puerto no aparece, revisar el driver CP210x o CH340.
- Mantener `T_REBOTE` entre 20 y 50 ms: valores menores pueden dejar pasar rebotes y valores mayores hacen sentir lento el pulsador.

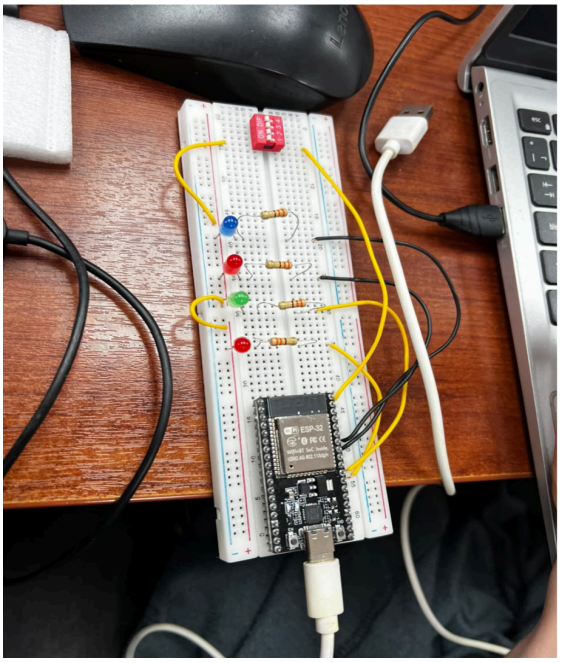
9. Bibliografía / Referencias

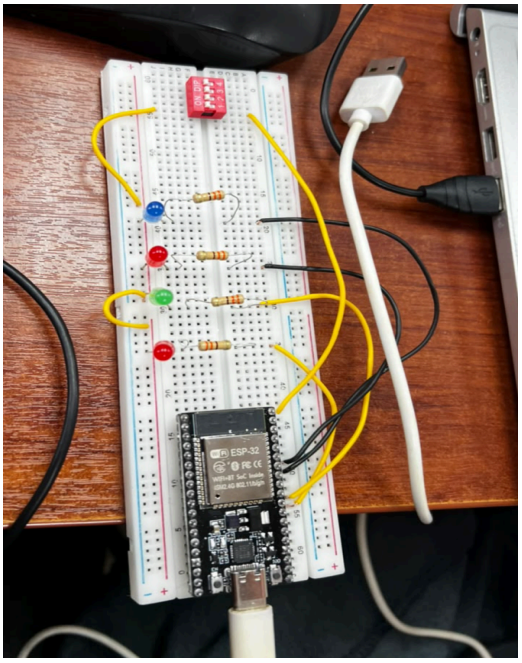
[1] L. A. Freire Sánchez, *Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico*, 2023, ISBN 978-9942-636-22-5.

[2] N. F. Zailani, *Getting Started with Arduino and ESP32: A Beginner's Guide with Wokwi*. Politeknik Mersing, 2024.

[3] Espressif Systems, "ESP32 Series Datasheet." [En línea]. Disponible: https://www.espressif.com/en/support/documents/technical-documents

10. Anexos





Declaración de uso de IA:

Para la presente práctica se utilizó la herramienta de IA de Claude con el fin de guiarnos durante el proceso del armado del circuito en físico para evitar dañar los componentes.